Class Diagram in UML

Unified Modeling Language

금오공과대학교
인공지능학과

학습 내용

- UML이란 무엇인가?
- UML Diagram 개요
- Class Diagram
- UML Tools

UML이란 무엇인가?

UML이란?

MODELING이란?

UML이란?

- UML
  - Standardized specification language for object modeling
  - 객체 지향적 분석 및 설계 방법론의 표준
    → 개발자 사이의 의사 소통 불일치 해소
  - 소프트웨어 시스템의 산출물을 명세화하고, 가시화하고, 구축하고, 문서화하기 위한 **시각적 모델링 언어**

- UML의 발전
  - 1997년 UML 1.0 by OMG (Object Management Group)
  - 2005년 UML 2.0
  - 2011년 8월 UML 2.4.1

MODELING이란?

- Model
  - 실체를 상상하거나 추측할 수 있도록 만든 대체 표현
    - 예) 패션모델, 프라모델, 모델하우스, 주가예측모델, …
  - 특징 및 유용성
    - 실체에 대한 정보 제공, 추론 구조 제공, 사고의 도구, 창조적 자극
  - 좋은 모델
    - 효과적 표현, 답을 향한 지도의 역할, 문제 풀이의 단서, 진화, 부분과 전체의 반영

UML Diagram

UML의 기본 관점

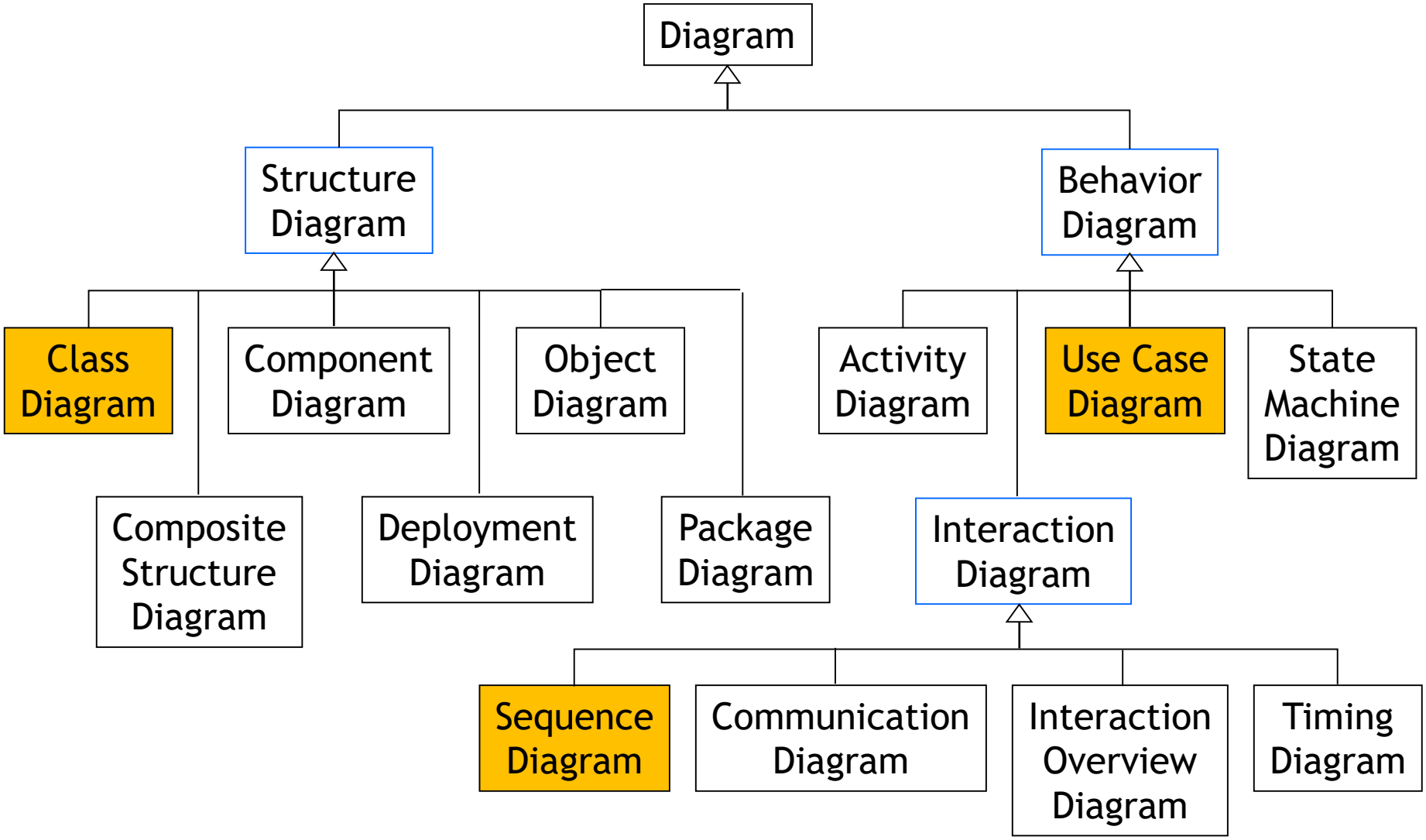
UML Diagram 종류

Class Diagram

UML의 기본 관점

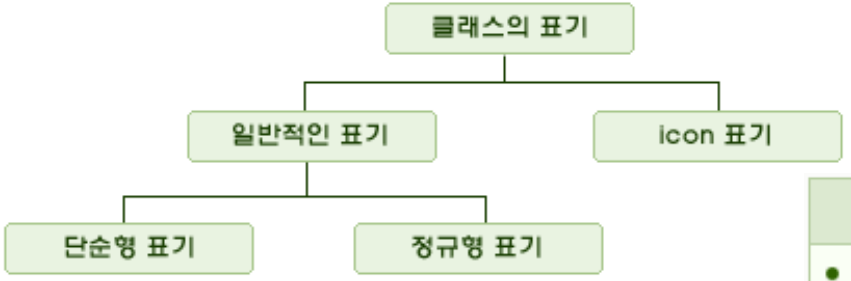
- Functional requirements view: 사용자 관점
  - 대표 diagram: Use case diagram
- Static structural view: 시스템의 정적 구조
  - Objects, attributes, operations, relationships
  - 대표 diagram: Class diagram, Composite structure diagram
- Dynamic behavior view: 시스템의 동적 행동
  - Object간의 협력 관계, Object 내부 상태 변경
  - 대표 diagram: Sequence diagram, Activity diagram, State machine diagram

UML Diagram 종류

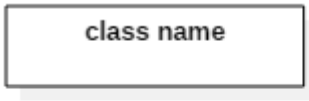


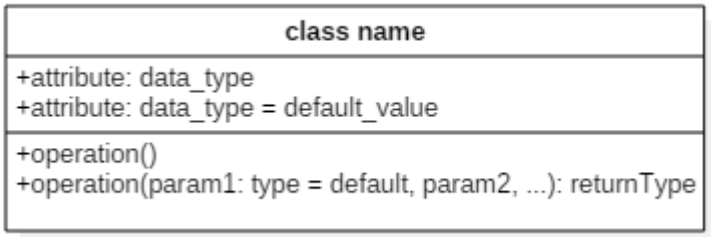
CLASS DIAGRAM : 개요

- 클래스 정의 및 클래스간의 관계를 나타냄
- 클래스: 속성attribute과 동작operation으로 표현

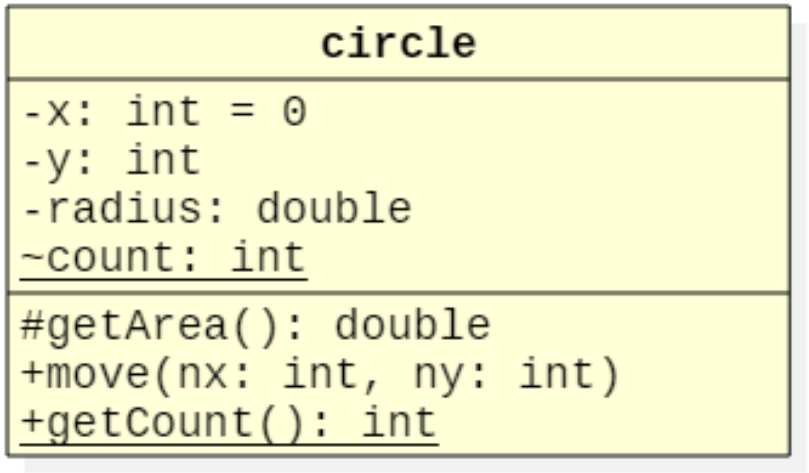


단순형 표기	정규형 표기
• 직사각형에 클래스 명을 표기 • Conceptual Level(개념 단계)의 클래스 다이어그램을 작성할 경우나, 클래스가 많아 그림이 복잡할 경우의 생략형 표현	• 세 개의 가로 간이 있는 사각형에 각각 클래스 명, 속성, 오퍼레이션을 차례로 표기





```
class circle {
   private int x=0, y;        // 중심 좌표
   private double radius;     // 반지름
   static int count;          // 객체 개수 저장

   protected double getArea();       // 면적 계산
   public void move(int nx, int ny); // 위치 이동
   public static int getCount();
};
```

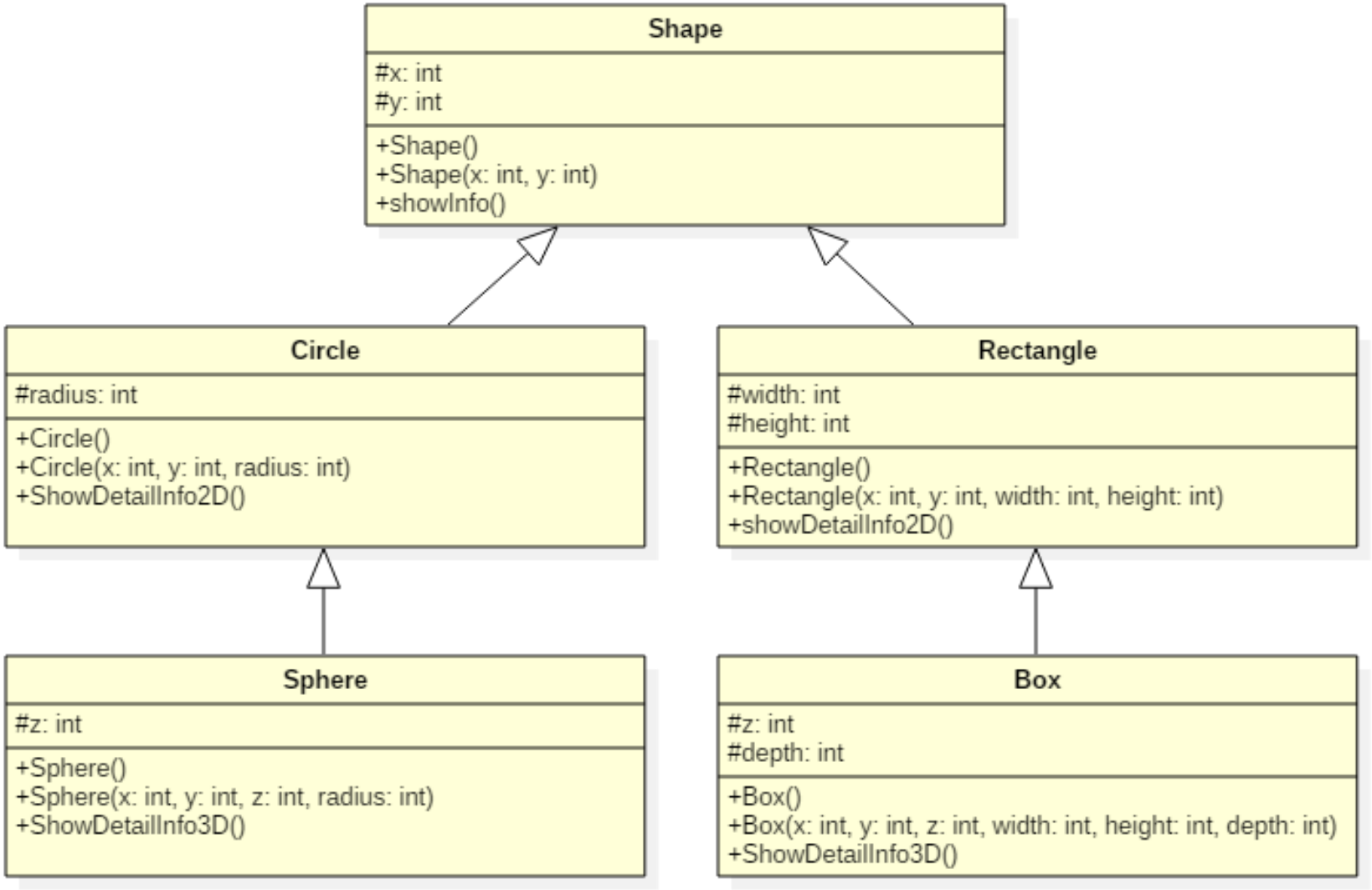


필드: 접근 지시자, 변수명, 타입, 초기값
메소드: 접근 지시자, 함수명, 매개변수, 반환형
접근 지시자
- : private
: protected
+ : public
~ : package
static : 밑줄

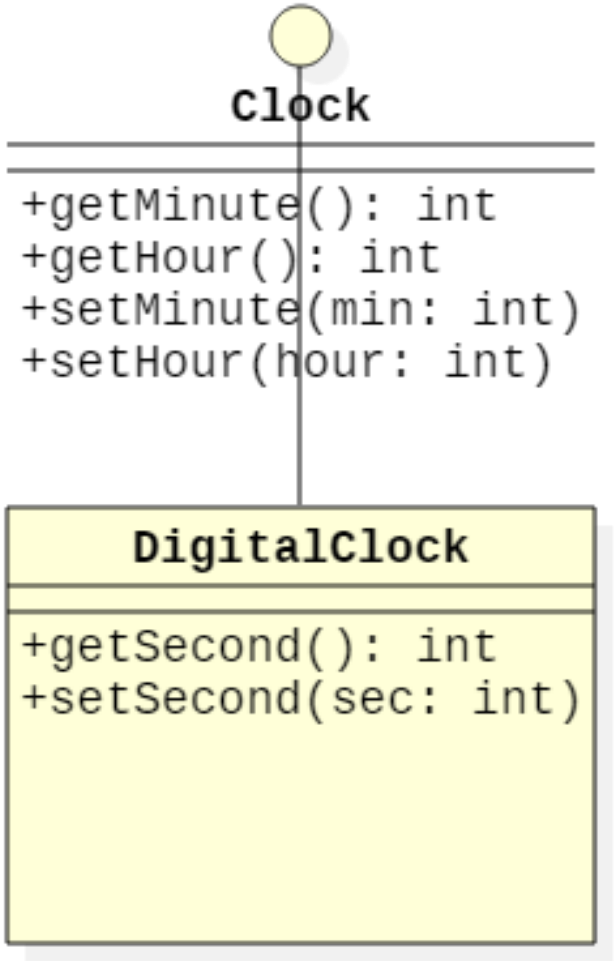
CLASS DIAGRAM : RELATIONSHIP

- 일반화(Generalization): is-a, 상속 관계
- 실체화(Realization): 인터페이스와 실체 클래스 (Java)
- 연관(Association): (주로) 참조 관계
  - 집합(Aggregation): has-a
  - 포함(Composition): 보다 강한 has-a 연관, life cycle 의존성
- 의존(Dependency): 한 클래스의 내용 변화가 다른 클래스에 영향을 미침

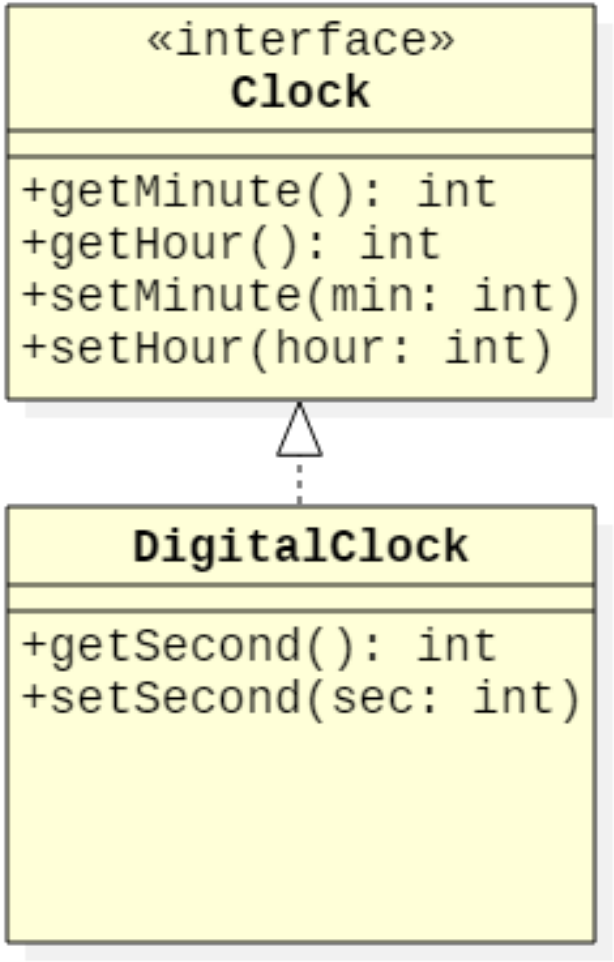
Class Diagram : Generalization



CLASS DIAGRAM : REALIZATION

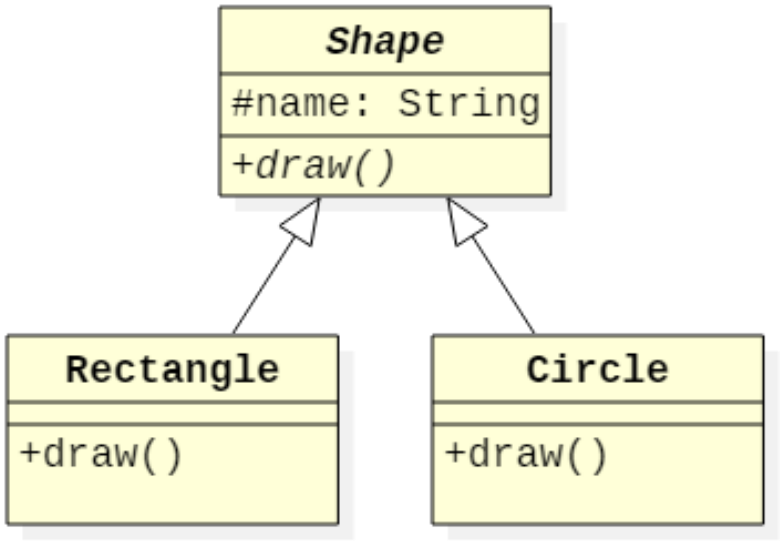


아이콘 표기법



스테레오 타입 사용

Class Diagram : Abstract Class




```
abstract class Shape {
  protected String name;

  public Shape() { name = null;}
  abstract public void draw();
}
```

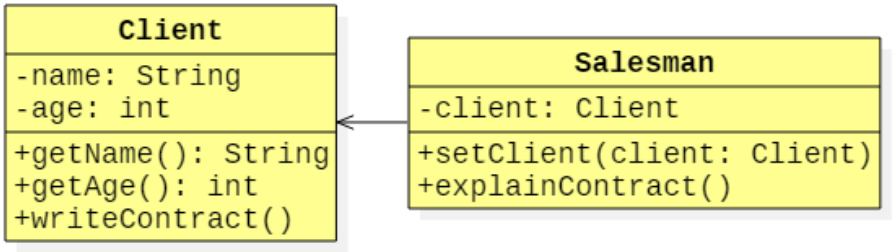
```
class Rectangle extends Shape {
  public void draw() {
     System.out.print("Rectangle");
  }
}
```

```
class Circle extends Shape {
  public void draw() {
     System.out.print("Circle");
  }
}
```

Class Diagram : Association (1)

일방 관계(Directed Association):

화살표를 사용하여 표현

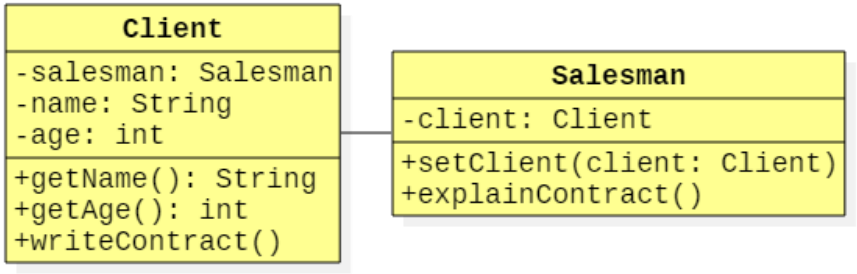



```
1  class Client {
2    private String name;
3    private int age;
4    public String getName() {
5      return this.name;
6    }
7    public int getAge() {
8      return this.age;
9    }
10   public void writeContact() {
11   }
12 }
13
14 class Salesman {
15   private Client client;
16   public void setClient(Client client) {
17     this.client = client;
18   }
19   public void explainContact() {
20     String name = this.client.getName();
21     int age = this.client.getAge();
22   }
23 }
```

Class Diagram : Association (2)

양방 관계(Association):

직선을 사용하여 표현

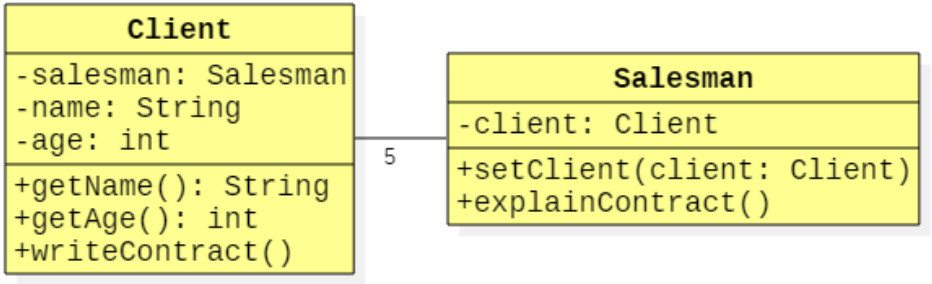



```
1  class Client {
2    private String name;
3    private int age;
4    private Salesman saleman;
5    public String getName() {
6      return this.name;
7    }
8    int getAge() {
9      return this.age;
10   }
11   void writeContact() {
12   }
13 }
14
15 class Salesman {
16   private Client client;
17   public void setClient(Client client) {
18     this.client = client;
19   }
20   public void explainContact() {
21     String name = this.client.getName();
22     int age = this.client.getAge();
23   }
24 }
```

Class Diagram : Association (3-1)

관계 수(multiplicity):

연관을 가지는 객체 수 표현

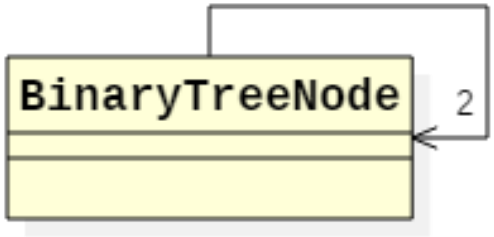



```
class Client {
  private String name;
  private int age;
  public String getName() {
    return this.name;
  }
  public int getAge() {
    return this.age;
  }
  public void writeContact() {
  }
}

class Salesman {
  private Client [] client = new Client[5];
  private int clientCount;
  public void manage(void) {
    for(int i = 0; i < ClientCount; i++) {
      string name = theClient[i].GetName();
      int age = theCliet[i].GetAge();
    }
  }
}
```

CLASS DIAGRAM : ASSOCIATION (3-2)

관계 수(multiplicity):

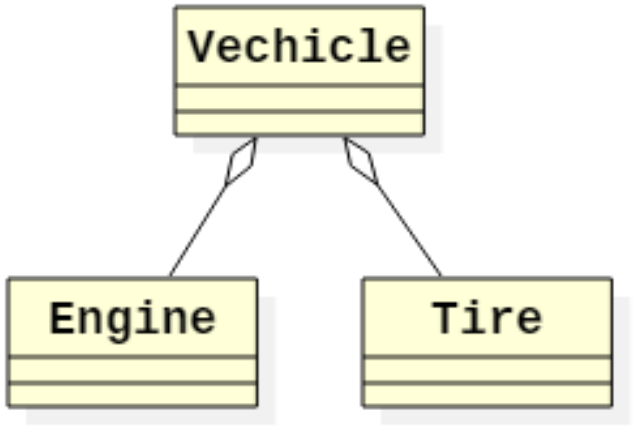
연관을 가지는 객체 수 표현




```
class BinaryTreeNode {
  private BinaryTreeNode leftNode;
  private BinaryTreeNode rightNode;
}
```

CLASS DIAGRAM : AGGREGATION

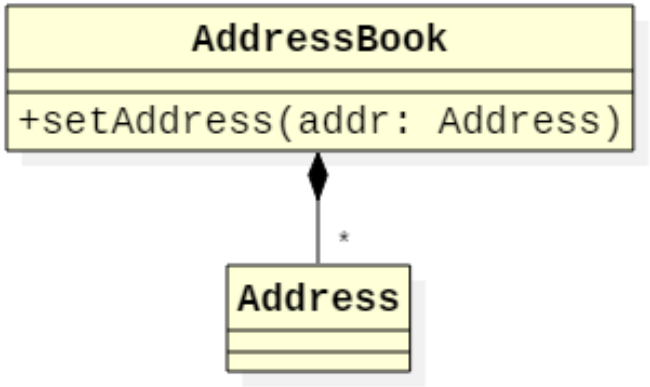
- "전체-부분(whole-part)" 관계일 경우 정의 (집합)
  - 클래스 각각이 독립적인 생명주기 가짐 (활용도 매우 낮음)




```
class Vechile {
  protected Engine itsEngine;
  protected Tire itsTire;
}
```

CLASS DIAGRAM : COMPOSITION

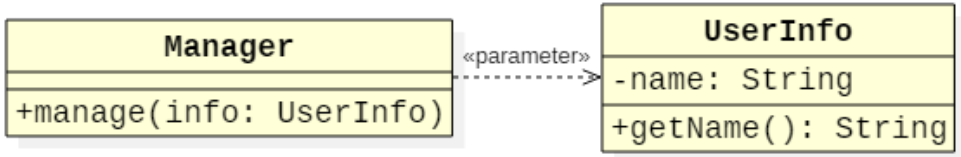
- "부분-전체"(part-of) 관계일 경우 정의 (합성)
  - 부분의 생명주기가 전체의 생명주기에 종속적인 관계



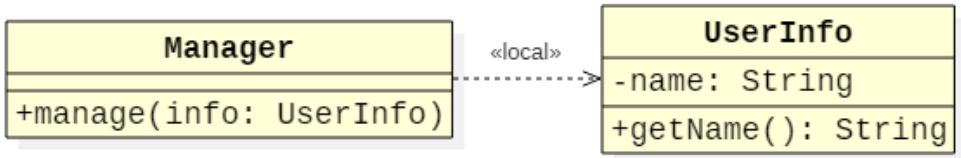

```
class AddressBook implements Cloneable {
  private Vector itsLine = new Vector();

  public void setAddress(Address addr) {
    itsLine.add(addr);
  }

  public AddressBook clone() {
    try {
      AddressBook clone =
          (AddressBook)super.clone();
      clone.itsLine = (Vector)itsLine.clone();
      return clone;
    }
    catch(CloneNotSupportedException e) {
      throw new AssertionError();
    }
  }
}
```

Class Diagram : Dependency

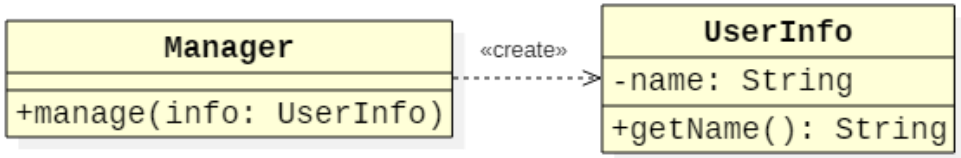
- 의존 관계가 발생하는 경우
  - 메소드의 파라미터에 사용
  - 메소드 내부의 지역 객체로 사용
  - 메소드 내부에서 객체 생성하여 반환




```
class Manager {
  public void manage(UserInfo info)
  {
    // use UserInfo
    string name = info.GetName();
  }
}
```




```
class Manager {
  public void use() {
    UserInfo info = new UserInfo();
    // use UserInfo
    string name = info.GetName();
  }
}
```




```
class Manager {
  public void create() {
    UserInfo info = new UserInfo();
    // return UserInfo
    return info;
  }
}
```

UML Tools

- StarUML
  - StarUML 2: http://staruml.io/
  - StarUML 1: http://staruml.sourceforge.net/ko/
  - 다이어그램 작성, Java/C++/C# 지원, 문서 작성, 패턴 지원 등 매우 다양한 기능 제공

- GitMind
  - https://gitmind.com/
  - 클래스 다이어그램, 시퀀스 다이어 그램 등 다양한 탬플릿 지원
  - 무료로 사용가능

UML 잘 사용하기

- 설명되지 않은 훨씬 많은 표기법과 구조가 있다!!
  - 매우 복잡하다.
  - 프로그램 외의 분야에서도 많이 사용된다.
  - 다 알 필요는 없다. 자신이 필요한 만큼, 이해하는 만큼만 사용하자.

- UML은 언제 어떻게?
  - 다른 사람들과 의사 소통
  - 문제점 해결을 위한 아이디어 창출
  - 로드맵
  - 보관 및 유지 보수, 개선을 위한 문서 (back-end)
    → 명세서

Hand Drawing

Tool 사용

Reference

- 황준하, “C++와 UML 기초”, 2009
- 윤현주, “UML 소개”, 2011
- 최은만, “소프트웨어 공학론”, 사이텍미디어, 2001
- 로버트 C. 마틴, “UML 실전에서는 이것만 쓴다 (Java 프로그래머를 위한 UML)”, 인사이트, 2004
- Craig Larman, “Applying UML and Patterns: An Introduction to object-Oriented Analysis and Design and the Unified Process, 3rd Ed.”, Prentice Hall, 2005